

Controlled Experiments Testing the Operator Account

Companion Report to “The Reasoning Mechanism of AI Neural Networks”

This report summarizes the controlled experiments behind the evidence section of the main paper, written to be readable without a background in interpretability. A point of method runs throughout: because these models are small and their wiring is known, we do not have to *guess* an operator from external tests; for the small cases we derive it *directly* from the weights as an explicit mathematical formula and check function- and topology-equivalence by derivation. Probes and causal interventions are used only as supporting checks, or where a network is too large to read by hand.

1. How evidence is graded

A few terms in plain language. A *cognitive neural operator* is a small computational subgraph that performs one nameable step (recognize, classify, compare, gate, suppress, transfer); it is defined by *what it computes*, not by which neurons host it. A *probe* is an auxiliary linear classifier testing whether a quantity can be read off a layer—it measures *materialization*, not operator presence. A *causal intervention* (ablation/patching) edits an internal value to see whether behaviour changes. *Strict accuracy* counts a case correct only if every output number matches within tolerance; because valid and invalid cases are very unequal in number we report them separately.

The grading, with the apex stated correctly. Contrary to the convention that puts a behavioural causal change at the top, a causal ablation is only a *qualitative* check that a part matters; it establishes neither the function nor the topology. We therefore grade as follows.

- **Basic / necessary checks.** (B1) the task is learned; (B2) a unit or edge tracks a condition, or a quantity is linearly readable (materialization); (B3) ablation/patching changes behaviour (a part is causally involved). These are necessary or corroborating, not sufficient.
- **Apex / quantitative operator evidence.** A computational subgraph is shown to be, *simultaneously*, (A1) **function-equivalent**—its closed form realizes the operator’s numerical mapping; (A2) **topology-matching**—its wiring is the operator topology or a defined variant; and (A3) **complete**—it realizes the whole cognitive step end to end (input condition $\rightarrow$ release/suppression). Only A1–A3 together establish that an operator is present; B3 alone does not.

For *category* operators we also note a cardinality observation (a class of N members is genuinely compressed only when $m < N$ control units span it), but—see Section 9—read by unit count it is basis-dependent and is used only as a coarse indicator, not a graded claim.

2. Criteria: functional equivalence and topological sameness

The object judged is the whole computational subgraph (boundary computation, value transfer, residual/intermediate state, ReLU-type gate, output read-out), not a single neuron or a probe score.

Functional equivalence is a process, not a truth table. The minimal skeleton is

$$d_{\text{wrong}} = \text{out-of-range evidence}, \quad Y = \phi(X_{\text{support}} - M d_{\text{wrong}} + b),$$

with recognition $R(X) = \phi(X - M \phi(\text{lo} - X) - M \phi(X - \text{hi}))$ (a category operator emits a constant m instead of X), where ϕ is any one-sided thresholded activation (Section 5). Two structures are functionally equivalent when they run the *same process*: a change in the input creates out-of-range evidence d_{wrong} ; that evidence enters a gate pre-activation with a *negative* weight; the pre-activation flips from positive to non-positive; the forward-support unit turns from a positive value to 0; and the conclusion is suppressed. The outputs are continuous values/vectors, *not* 0/1 truth values (recognition emits the concept value inside its range and 0 outside; classification a category constant; comparison $|A_1 - A_2|$; release the payloads C, B). Equivalence is established by reading this closed form, and confirmed where needed by matching value mappings and by causal interchangeability.

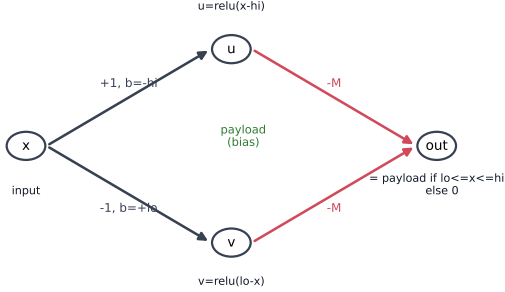
Topology: same, or a defined variant. The standard recognition topology has three parts: a forward-support path, an out-of-range evidence path, and a negative-weight suppression path that acts *before* the truncation. Two wirings have the *same* topology when these three parts correspond one-to-one. They are a *variant* (same function, reorganized wiring) when related by: **distributed** (spread over many units), **split** (sub-conditions separated), **cross-shared** (sharing units with other operators), **embedding-folded** (part pre-encoded in embeddings), **reverse-detection** (reads the out-of-range evidence and closes the valid path), or **stretched** (a value carried by an identity / near-identity map through extra layers). The distinction we draw is: *same topology* = three paths in one-to-one correspondence; *similar / variant topology* = the same three functional roles present but realized by one of the reorganizations above. Counting active neurons is therefore not a topology test; a single functional path can be distributed horizontally and stretched vertically and still be the same operator.

3. Operator signatures and structure diagrams

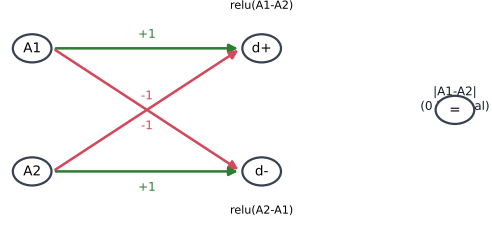
Figure 1 gives the closed-form/geometric fingerprint of each operator: recognition (a payload passed inside a value band, suppressed outside by two boundary detectors); comparison (antisymmetric difference, the A_1 weight block equal to minus the A_2 block); and the two gate

forms—bias-filling ($\text{ReLU}(\text{payload} + w_c x_c - B)$, a deep negative bias opened by a condition) and negative-weight blocking ($\text{ReLU}(\text{payload} - M d_{\text{wrong}}$, a path closed by a mismatch).

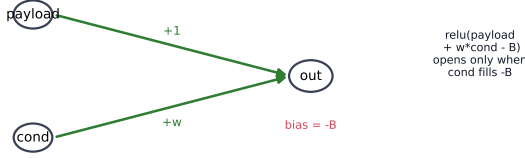
(a) Recognition / interval operator



(b) Comparison / antisymmetric difference



(c) Gate form 1: bias-filling



(d) Gate form 2: negative-weight blocking

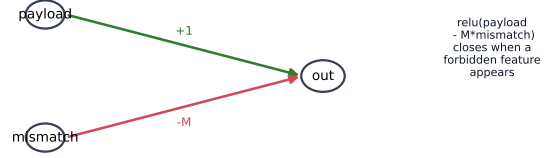


Figure 1: Operator structure diagrams. Green = positive (support) weights; red = negative (suppression/blocking) weights. (a) recognition; (b) comparison; (c,d) the two gate forms.

4. Explicit construction and direct readout

The hand-specified circuit is checked by exhaustive simulation: over its full input space it reproduces the target transformation, and the upstream word/phrase-to-concept-to-circuit pipeline passes all 16,680/16,680 tests, so the mapping from surface expressions through recognition and concept generation into the reasoning core is exact, not approximate (Figure 2). It releases $[C, 3, B]$ for every valid premise pair and suppresses to $[0, 0, 0]$ for every violation of relation, quantifier, predicate, or middle-term match.

Because the core is small and sparse, every node reads out as an explicit operator (apex A1–A3 by inspection):

$$\begin{aligned}
 \text{pass_C_if_relation} &= \text{ReLU}(C + 2.857 \cdot \text{belongs_cat} - 20) && \text{bias-filling gate} \\
 \text{A2_minus_A1} &= \text{ReLU}(-A_1 + A_2), \quad \text{A1_minus_A2} = \text{ReLU}(A_1 - A_2) && \text{comparison} \\
 \text{pass_C_if } A_1=A_2 &= \text{ReLU}(C - 20(A_2 - A_1) - 20(A_1 - A_2)) && \text{blocking gate}
 \end{aligned}$$

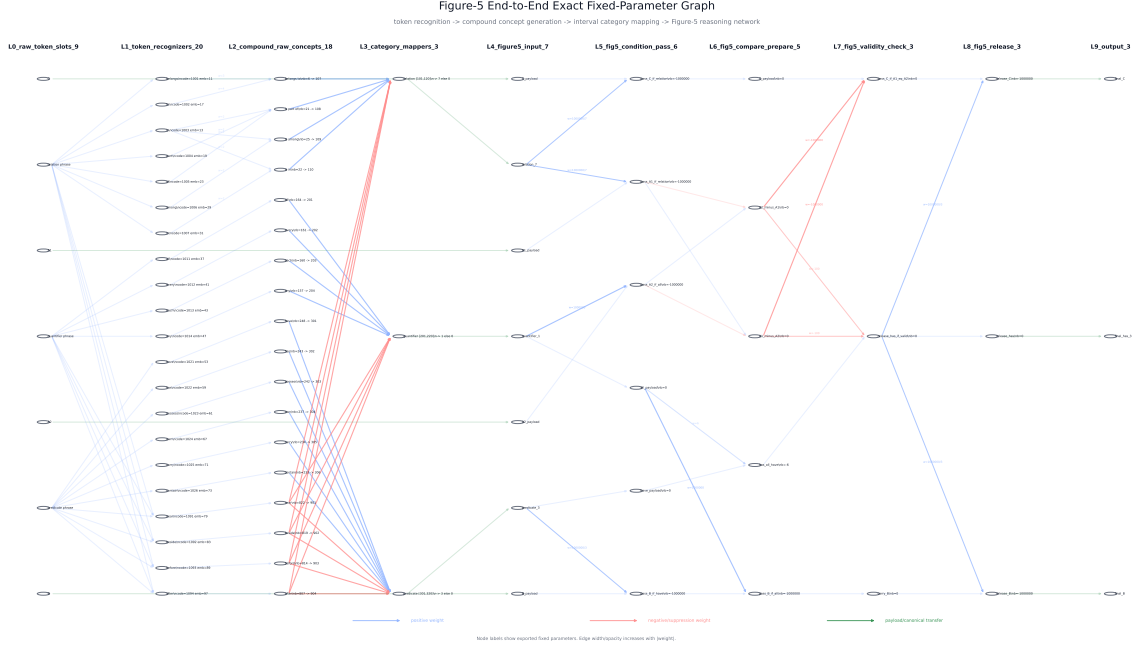


Figure 2: End-to-end explicit fixed-parameter graph: token recognition, compound concept generation, interval category mapping, and the reasoning core, each on its own nodes.

5. Activation functions: do other valves equal the ReLU valve?

The recognition operator needs an upper-boundary and a lower-boundary detector plus a suppression—a one-sided “valve.” ReLU is the sharpest such valve, but it is not the only one. We ask directly whether SiLU, GELU, Softplus (and attention scores) realize a *functionally equivalent* valve, and we derive the equivalence rather than assert it.

Derivation. For any smooth activation ϕ with $\phi(z) \rightarrow 0$ as $z \rightarrow -\infty$ and $\phi(z) \approx z$ as $z \rightarrow +\infty$, the gain-scaled map $\phi_a(z) = \phi(az)/a$ satisfies $\phi_a(z) \rightarrow \text{ReLU}(z)$ pointwise as $a \rightarrow \infty$. Building the interval valve with ϕ_a in place of ReLU therefore reproduces the same numerical mapping in the high-gain limit: the boundary detector $\phi_a(x - \text{hi})$ is a soft threshold that sharpens to $\text{ReLU}(x - \text{hi})$, and the suppressed output converges to the ideal recognition function. This is functional equivalence in the sense of Section 2 (same value mapping), differing only by a *soft* versus *sharp* boundary; the topology (three paths) is identical.

Numerical check. Table 1 measures the realized interval valve $R(x) = \text{val}$ if $\text{lo} \leq x \leq \text{hi}$ else 0, built with each activation, against the ideal, away from the boundary transition. ReLU is exact; SiLU, GELU and Softplus converge to it as the gain grows. Figure 3 shows the valves overlapping the ideal and the boundary detector $\phi_a(z) \rightarrow \text{ReLU}(z)$. So the mechanism depends on *one-sided thresholded computation*, not on ReLU specifically. Attention contributes the same way: a score $s = Q \cdot K = aX + b$ is the boundary function $aX + b$ used by the detectors, so attention can supply the boundary/out-of-range evidence part, with a downstream ReLU doing the truncation (Section 7).

activation	gain $a=1$	$a=3$	$a=10$	$a=30$
ReLU	0.000	0.000	0.000	0.000
SiLU	21.5	4.31	0.95	0.019
GELU	12.7	2.27	0.179	0.000
Softplus	3.00	3.00	0.504	0.003

Table 1: Max deviation of the realized interval valve from the ideal (away from the boundary transition), versus gain a . ReLU is exact; smooth activations converge to the same valve.

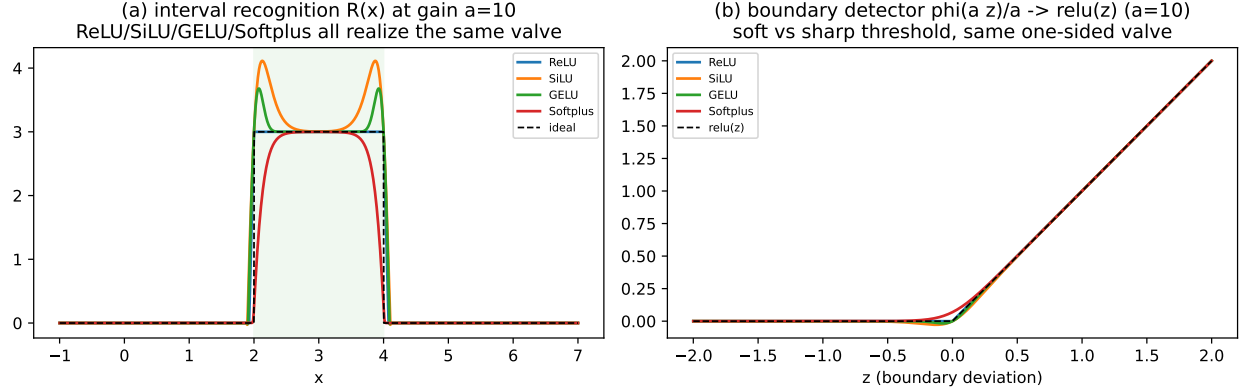


Figure 3: (a) the interval recognition operator realized by ReLU/SiLU/GELU/Softplus (gain $a=10$) all track the ideal; (b) the boundary detector $\phi_a(z) \rightarrow \text{ReLU}(z)$: a soft versus sharp threshold, the same one-sided valve.

6. Distributed topology: a derivation, not an assertion

The claim that a wide network realizes an operator as a *distributed* structure is made precise here. ReLU is positive-homogeneous, $\text{ReLU}(wz) = w \text{ReLU}(z)$ for $w > 0$. Hence K units $\{\text{ReLU}(w_k z)\}_{k=1}^K$ that share a direction z (e.g. $z = A_1 - A_2$ for comparison), read out by weights $\{v_k\}$, compute

$$\sum_{k=1}^K v_k \text{ReLU}(w_k z) = \left(\sum_{k=1}^K v_k w_k \right) \text{ReLU}(z),$$

which is the single-unit operator $\text{ReLU}(z)$ when $\sum_k v_k w_k = 1$. We verify numerically with $K=5$ random w_k and a matched read-out: over a grid the distributed structure equals the single unit to machine precision ($\max |\cdot| \approx 9 \times 10^{-16}$, $\sum_k v_k w_k = 1.000$). Figure 4 draws the two topologies side by side. This is the sense in which the wide-network “distribution” of the comparison operator (Section 9) is function-equivalent: the same operator, its single path spread over many units.

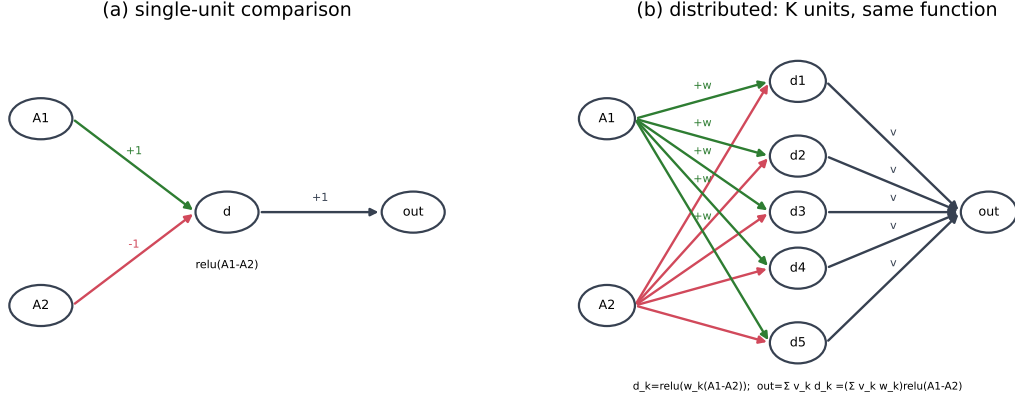


Figure 4: (a) single-unit comparison $\text{ReLU}(A_1 - A_2)$; (b) a K -unit distributed realization $\sum_k v_k \text{ReLU}(w_k(A_1 - A_2)) = (\sum_k v_k w_k) \text{ReLU}(A_1 - A_2)$. Same function, distributed topology.

7. Transformer correspondence

On a standard attention-plus-ReLU Transformer trained on the task, we observe the same operator families the theory predicts—recognition, classification, comparison, and the gate forms—realized as topological variants (distributed / shared-gating) of the operator chain. The correspondence holds at the level of operator roles and their functional behaviour and matches the theory’s prior predictions. The specific procedure we use to locate and decompose these operators inside a Transformer is proprietary and is not expanded here (commercial-confidential information); this report states the observation only, not the method or the internal wiring detail.

8. Reading function- and topology-equivalence out of a trained network

We now show, by derivation, how a *trained* (not constructed) network is judged function-equivalent and topology-equivalent. For each operator we read three views directly from the trained weights: the *function* learned (target vs trained output), the *topology* (each neuron’s explicit formula and read-out weight drawn on the graph), and the *hidden units* (each unit’s activation curve).

Equality gate (Figure 5). A tiny network trained only on the equality task (output P if $A_1 = A_2$, else 0; ReLU; final-output supervision) reaches strict accuracy 1.0, and its hidden units read out as two antisymmetric difference units $\text{ReLU}(w(A_1 - A_2))$ and $\text{ReLU}(w(A_2 - A_1))$ that enter the output with large *negative* weights (the suppression M), while the payload P is carried by positive-weight units. **Function-equivalence (A1):** the realized map is $\text{ReLU}(P - M |A_1 - A_2|)$. **Topology (A2):** the three roles are present—forward-support (P), out-of-range evidence ($|A_1 - A_2|$), negative-weight suppression before the output ReLU—a (slightly distributed) variant of Figure 1(b,d). **Complete (A3):** input→evidence→suppression→release runs end to end at strict accuracy 1.0.

Interval recognizer (Figure 6). Network 131 trained to emit the category inside $[0.3, 0.7]$

and 0 outside reads out as a lower-boundary detector $\text{ReLU}(+ax+b)$ and an upper-boundary detector $\text{ReLU}(-ax+b')$ feeding the output with negative weights—the recognition skeleton $\text{ReLU}(\text{val} - M \text{ReLU}(\cdot) - M \text{ReLU}(\cdot))$ of Figure 1(a). In both cases the judgement is made by reading the arithmetic, not by probing.

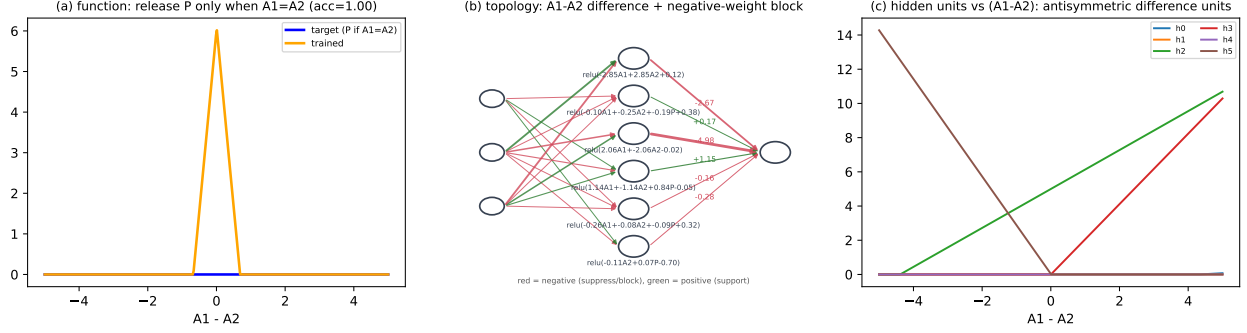


Figure 5: Direct readout of a trained equality gate. (a) function: P is released only at $A_1=A_2$; (b) topology: each neuron’s formula and read-out weight, showing the antisymmetric difference units entering with large negative (red) weights and the payload on a positive (green) weight; (c) hidden units versus $A_1 - A_2$: the two antisymmetric difference units.

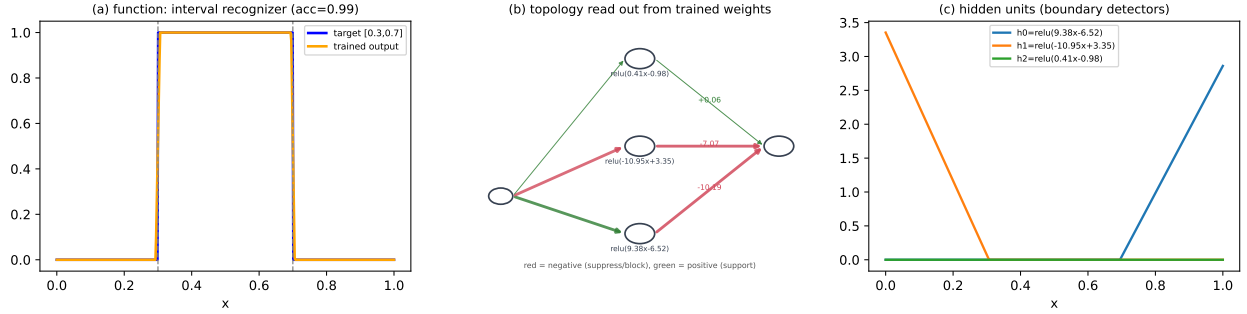


Figure 6: Direct readout of a trained interval recognizer. (a) function: the trained output matches the target band; (b) topology with each neuron’s formula; (c) hidden units = boundary detectors. The recognition skeleton, read directly from the parameters.

Trained end to end with only the final answer supervised, a network’s concept representation also organizes geometrically (Figure 7): synonyms collapse to one value (classification), roles separate into bands (the category intervals), multi-token relations stay composed—read directly from the parameters.

Assessment of the staged-process prediction. The explicit parameter construction and the masked networks strongly verify the theory’s staged process, because each stage has a definite mathematical function and parameter location:

token recognition $\rightarrow$ phrase composition $\rightarrow$ category normalization
 $\rightarrow$ comparison $\rightarrow$ gate $\rightarrow$ suppression/release.

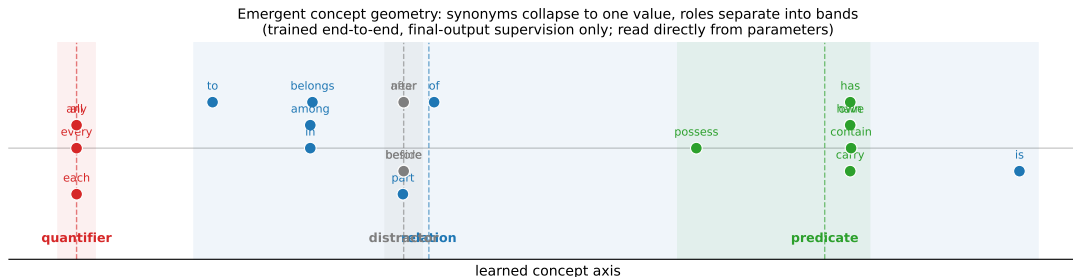


Figure 7: Emergent concept geometry, read directly from a trained network: synonyms collapse to one value, roles separate into bands, multi-token relations remain composed.

These stages are not named post hoc; they can be read out layer by layer from the weights, biases, and connection paths. Hence, within the controlled small-model scope, the experiments have shown this language-reasoning staged process to be realizable, verifiable, and preserved under training.

9. Operators across multiple dimensions: same function, distributed topology

Trained densely without masks across a range of embedding widths, the task is solved at every width, so capability is not the variable. The validity gate is cleanly linearly readable at every dimension; the interior recognition and comparison conditions are present but distributed, recovered by nonlinear probes where the linear probe is weak (a topological variant, not a missing operator).

The “not fully antisymmetric” comparison, explained (no theory conflict). Read at the parameter level, the comparison operator is a clean antisymmetric $A_1 - A_2$ difference unit at low dimension (the A_1 block equals minus the A_2 block, cosine ≈ 1). At high dimension *no single unit* carries it; instead the antisymmetric weight is spread across many units, each contributing a fraction (this is the K -unit distributed realization derived in Section 6, where $\sum_k v_k w_k$ reconstructs the single unit). This is *not* a training failure and *not* a conflict with the theory: by Section 6 the distributed structure computes the same $\text{ReLU}(A_1 - A_2)$ function; what drops is only the *single-unit linear readability*, not the operator. Read at the representation level the decision stays low-rank and linearly recoverable at every dimension, with no unused embedding dimensions—what grows with width is the operator’s *footprint across units*, not its intrinsic dimensionality.

10. Necessity and generalization: structure is what makes reasoning generalize

What does explicit operator structure buy? The answer is the core value of the account, and must be stated correctly: **operator structure is what gives the model genuine, generalizing reasoning.** A dense network can *fit* a task by accumulating many units

into a direct input–output lookup—memorization—but this brings no generalization and an explosion in parameters; it is not reasoning. The operator structure, by reusing a small set of computational primitives, performs the reasoning compactly and extrapolates to new cases.

The controlled evidence: in a factorial ablation, only theory-structured networks form the clean staged chain, while dense/random variants—even with full intermediate supervision—collapse to strict accuracy below 0.34 while still classifying valid versus invalid, the signature of a memorized shortcut that has *not* formed the operators. In compositional generalization (Table 2), a dense MLP reaches near-zero out-of-distribution accuracy on chain-length and held-out-template splits, while a shared-operator network and a fixed-theory operator generalize almost perfectly. An independent multi-step replication shows the same direction: a weight-shared operator-reuse network generalizes to unseen chain lengths (0.994) while a same-shape dense network fits in distribution (1.000) but drops on the unseen lengths (0.704). Structure thus buys reusable, separable, auditable operators that generalize—the very thing memorization cannot provide.

model	chain-length OOD	held-out-template OOD
dense MLP	0.010	0.000
shared-operator network	0.973	1.000
fixed-theory operator	1.000	1.000

Table 2: Out-of-distribution strict accuracy. Structured operators generalize; the memorizing dense map does not.

11. Hallucination as a gate that leaks

The account predicts that as the validity gate (an AND of conditions) weakens it degrades toward an OR and releases conclusions on insufficient evidence. We weaken the gate progressively (steering invalid cases toward the valid decision state by a fraction β) and measure how often invalid cases are falsely released. The rate rises monotonically and sharply: near zero while the gate holds, then releasing essentially all invalid cases once sufficiently weakened (1.00 ± 0.00 at $\beta = 0.8$, five seeds). Hallucination is thus a concrete, measurable gate failure, with concrete remedies (tighten recognition ranges, raise gate thresholds, strengthen negative-weight blocking).

12. The discriminating (falsification) test, explained

What it is for. We want to be sure the operators respond to the *real* task conditions and not to some coincidental feature that happens to correlate with the answer. A test that could only ever “pass” proves little; a useful test is one that *could have failed* if the operator account were wrong. Here is such a test.

Design. We train networks whose validity rule is the AND of k of the four conditions (relation, quantifier, predicate, middle-term match), for $k = 1, 2, 3, 4$. By construction we know which conditions *should* matter and which should not.

Two predictions that could have failed.

- **Only required conditions suppress.** Violating a condition that *is* in the rule drives the output to ≈ 0 (output ratio 0.002); violating a condition that is *not* in the rule leaves the output essentially unchanged (ratio 1.000). If the network had keyed on a spurious correlate, an irrelevant violation would also have suppressed the output—it did not.
- **The gate is conjunction-driven.** The number of distinct violation types the shared gate responds to tracks k (0.67, 2.0, 2.85, 3.83 for $k = 1, 2, 3, 4$). A $k=1$ task does not manufacture a spurious multi-condition gate; a multi-condition gate forms only when the task truly conjoins conditions.

How this “falsifies”. Either outcome going the other way (an irrelevant violation suppressing the output, or a single-condition task building a multi-condition gate) would have shown the operators do *not* track the real task logic, refuting the operator-correspondence claim. Both held, so the claim survives a test it could have failed—which is what makes it evidence rather than restatement. The same selectivity is visible directly (Figure 8): under controlled one-condition-at-a-time contrasts, each required condition drives its own activation contrast and the full validity gate dominates—the operators respond to the real task conditions, not to a spurious correlate.

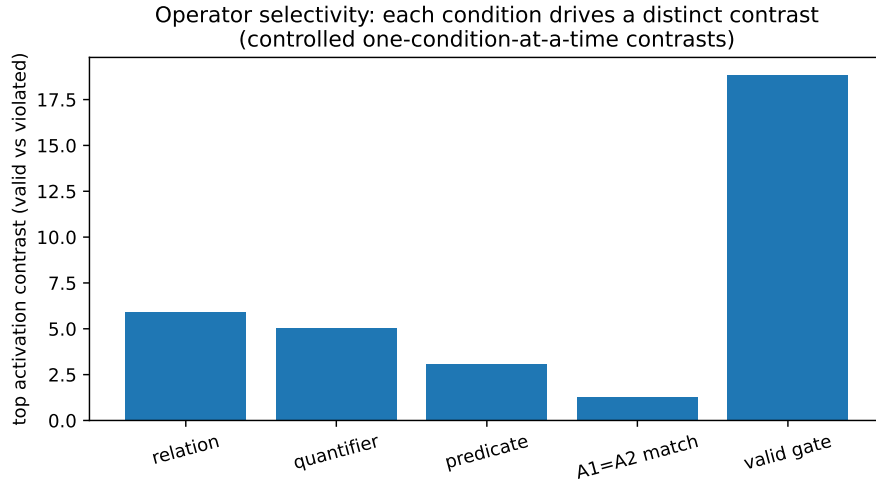


Figure 8: Operator selectivity by controlled contrast: the change in the top unit’s activation when a single condition is violated. Each condition has its own contrast and the full validity gate is largest—evidence the operators track the actual conditions.

13. Claim-by-claim evidence

Table 3 grades each claim. ✓ = solid; ~ = partial / with stated caveat. “Read” marks claims established by direct closed-form derivation (apex A1–A3).

14. Methods: training, loss, and metrics

For completeness: the freely trained networks are supervised on the final output only (no intermediate targets) with a class-balanced relative mean-squared error $\ell = \langle w ((\hat{y} - y)/(|y| + 1))^2 \rangle$, where w up-weights the rare valid class; optimizer AdamW (learning rate 3×10^{-3} , cosine schedule); strict accuracy is the fraction of cases with $\max_j |\hat{y}_j - y_j| < 0.35$, reported split by valid/invalid. The masked networks use the same recipe with a fixed sparse connectivity mask read from the construction. The activation and distributed analyses (Sections 5–6) are analytic/numerical and require no training. Models, logs, and the per-experiment configurations are saved alongside this report.

15. What the experiments establish

The operator families—recognition, classification, comparison, the two gate forms, suppression, transfer, concretization—are realizable at the parameter level, exhaustively verifiable, and, for the small cases, *derived directly* from the weights as function- and topology-equivalent structures rather than inferred from external tests. Smooth activations and attention realize the same valves (derived in the high-gain limit); a wide network’s distributed realization is shown by derivation to compute the same operator; and a standard Transformer is observed to realize the same operator families as topological variants, confirming the theory’s prior predictions. Explicit operator structure is what gives the model *generalizing* reasoning—reusable, separable, auditable operators that extrapolate—whereas an unstructured dense network can only memorize, at the cost of generalization and parameter count. Hallucination appears as a measurable gate leak, and a discriminating test shows the operators track the real task conditions. The principal limit is scope: this work is on small, controlled models; **we have not yet carried out operator-extraction experiments on commercial large models**—that work is simply not yet begun.

claim	evidence	status
Recognition / classification operators	read directly (A1–A3)	✓
Concept generation & normalization	verified (synonyms $\rightarrow$ one value)	✓
Two gate forms (bias-filling, blocking)	read directly	✓
Comparison = antisymmetric difference	read (construction + trained net); distributed at high width (Sec. 6,9)	✓
Full syllogism verified end to end	exhaustive, 16,680/16,680	✓
Activation equivalence (ReLU/SiLU/GELU/attention)	derived: same value in high-gain limit	✓
Distributed topology = same operator	derived ($\sum_k v_k w_k \text{ReLU}(z)$) + verified	✓
Transformer correspondence	observed operator-topology variants; method confidential	✓
Masked full chain trains to strict 1.0	all cases pass, read back as the chain	✓
Free dense nets realize operators as variants	read + probe + causal	✓
Structure $\Rightarrow$ generalizing reasoning; dense memorizes (no generalization)	OOD controls (Table 2) + multi-step	✓
Hallucination = measurable gate leak	causal weakening curve	✓
Falsifiable AND-gate (discriminating test)	two predictions that could have failed	✓
Multi-dimensional realization	decision low-rank & readable at all; footprint spreads	~
Large-model phenomena correspondence	interpretive prediction; no commercial-model experiment yet	~

Table 3: Claim-by-claim evidence. The strongest evidence for the small cases is direct function- and topology-equivalence read from the parameters (apex A1–A3); probes and causal tests are supporting checks.